

Lecture 16: Training transformer models

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain what it means to "train" a transformer model
2. Describe the training loop: forward pass, loss, backward pass, update
3. Understand cross-entropy loss and perplexity as training metrics
4. Explain scaling laws and their practical implications
5. Distinguish between pre-training and fine-tuning

Where we left off

Recap from lecture 15

Last time we saw the transformer as a function:

$$\text{Transformer}(X, \theta) \rightarrow Y$$

We traced data from raw text through tokenization, embedding, attention, and feed-forward layers. But we treated the parameters θ as given.

Today's question

Where do the parameters θ come from? How does the model *learn* to predict the next token?

What does "training" mean?

Training a language model

Training is the process of finding parameters θ that make the model's predictions match the data. Given a large corpus of text, we want:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

where $\mathcal{L}$ is a **loss function** that measures how wrong the model's predictions are.

Intuition

Imagine the model is a student learning to finish sentences. We show it millions of sentences, and every time it guesses the next word wrong, we nudge its parameters to make a better guess next time.

The language modeling objective

Next-token prediction

Given a sequence of tokens $x_1, x_2, \dots, x_{t-1}$, the model predicts a probability distribution over the vocabulary for the next token x_t :

$$P(x_t \mid x_1, x_2, \dots, x_{t-1}; \theta)$$

The goal is to maximize the probability of the *actual* next token across the entire training corpus.

This is self-supervised learning

No human labels needed! The "label" is simply the next word in the text. This is why we can train on virtually unlimited data from the internet.

Cross-entropy loss

The loss function

Cross-entropy loss measures how different the model's predicted distribution $\hat{y}$ is from the true distribution y (a one-hot vector for the correct token):

$$\mathcal{L} = - \sum_{i=1}^V y_i \log(\hat{y}_i)$$

Since y is one-hot (only the correct token has $y_i = 1$), this simplifies to:

$$\mathcal{L} = -\log(\hat{y}_c)$$

where c is the index of the correct next token.

Understanding cross-entropy

Why negative log probability?

- If the model assigns probability 1.0 to the correct token: $-\log(1.0) = 0$ (no loss!)
- If the model assigns probability 0.5: $-\log(0.5) = 0.69$
- If the model assigns probability 0.01: $-\log(0.01) = 4.6$ (high loss!)

The loss penalizes confident *wrong* predictions heavily and rewards confident *correct* predictions.

Concrete example

```
1 Vocabulary: [the, cat, sat, on, mat] (V=5)
2 True next token: "sat" → y = [0, 0, 1, 0, 0]
3
4 Model prediction:  $\hat{y} = [0.1, 0.2, 0.5, 0.15, 0.05]$ 
5 Loss =  $-\log(0.5) = 0.69$ 
6
7 Better prediction:  $\hat{y} = [0.05, 0.05, 0.85, 0.03, 0.02]$ 
8 Loss =  $-\log(0.85) = 0.16$  ← much lower!
```

Perplexity

Perplexity: a more intuitive metric

Perplexity is the exponential of the average cross-entropy loss across a sequence:

$$\text{PPL} = e^{\mathcal{L}} = e^{-\frac{1}{N} \sum_{t=1}^N \log P(x_t | x_{<t})}$$

Intuition

Perplexity measures how "confused" the model is. A perplexity of k means the model is, on average, as uncertain as if it were choosing uniformly among k options.

- **PPL = 1**: Perfect prediction (impossible in practice)
- **PPL = 10**: Like choosing among 10 equally likely words
- **PPL = 100**: Very uncertain
- **GPT-3 on test data**: PPL ≈ 20

The training loop

Four steps, repeated millions of times

1. **Forward pass:** Feed a batch of text through the model to get predictions
2. **Compute loss:** Compare predictions to actual next tokens using cross-entropy
3. **Backward pass:** Compute gradients $\nabla_{\theta}\mathcal{L}$ via backpropagation
4. **Update parameters:** Adjust θ to reduce the loss

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

where η is the **learning rate** — how big a step we take.

Scale matters

GPT-3 has 175 billion parameters. Each training step updates *all* of them. Training took ~\$4.6 million in compute costs and processed ~300 billion tokens.

Gradient descent intuition

The hiking analogy

Imagine you're lost in a foggy mountain range and want to reach the lowest valley:

1. **Feel the slope** under your feet (compute the gradient)
2. **Take a step downhill** (update parameters in the direction that reduces loss)
3. **Repeat** until you reach a valley (convergence)

The **learning rate** (η) controls your step size:

- Too large $\rightarrow$ you overshoot the valley and bounce around
- Too small $\rightarrow$ you make progress painfully slowly
- Just right $\rightarrow$ you descend efficiently

Stochastic gradient descent

In practice, we don't compute the gradient over the *entire* dataset (too expensive). Instead, we use random **mini-batches** of data — this is called **stochastic gradient descent** (SGD).

Optimization: AdamW

The optimizer of choice for transformers

AdamW (Adaptive Moment Estimation with Weight Decay) improves on basic gradient descent:

1. **Adaptive learning rates:** Each parameter gets its own learning rate based on gradient history
2. **Momentum:** Uses exponential moving average of past gradients to smooth updates
3. **Weight decay:** Adds regularization to prevent parameters from growing too large

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momentum})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{squared gradients})$$

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} - \lambda \theta_{t-1} \quad (\text{update + decay})$$

Why AdamW?

Standard Adam "hides" weight decay inside the adaptive learning rate, making it less effective. AdamW decouples weight decay from the gradient-based update, leading to better generalization.

Learning rate scheduling

Warmup then decay

Modern transformers use a **learning rate schedule** with two phases:

1. **Warmup** (first ~1-10% of training): Gradually increase η from 0 to the target value. This prevents large, unstable updates early when the model is randomly initialized.
2. **Decay** (remaining training): Gradually decrease η using a cosine or linear schedule. This allows fine-grained optimization as the model approaches convergence.

Learning rate schedule in Python

```
1 from transformers import get_cosine_schedule_with_warmup
2
3 optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
4
5 scheduler = get_cosine_schedule_with_warmup(
6     optimizer,
7     num_warmup_steps=1000,      # warmup for 1000 steps
8     num_training_steps=100000  # total training steps
```

continued...

Learning rate scheduling

```
9     )
10
11     # In training loop:
12     for batch in dataloader:
13         loss = model(batch).loss
14         loss.backward()
15         optimizer.step()
16         scheduler.step() # update learning rate
```

...continued...

Learning rate scheduling

17 | `optimizer.zero_grad()`

...continued

Gradient clipping

The exploding gradient problem

During backpropagation through many layers, gradients can grow exponentially large — this is called **exploding gradients**. A single bad gradient can destroy hours of training progress by making a catastrophically large parameter update.

The fix: gradient clipping

Before updating parameters, cap the total gradient norm to a maximum value:

$$\text{if } \|\nabla_{\theta} \mathcal{L}\| > \text{max_norm} : \quad \nabla_{\theta} \mathcal{L} \leftarrow \text{max_norm} \cdot \frac{\nabla_{\theta} \mathcal{L}}{\|\nabla_{\theta} \mathcal{L}\|}$$

This preserves the gradient *direction* while limiting its *magnitude*. A common choice is $\text{max_norm} = 1.0$.

Training with HuggingFace

A complete training loop

```
1  from transformers import AutoModelForCausalLM,  
   AutoTokenizer, Trainer, TrainingArguments  
2  from datasets import load_dataset  
3  
4  # Load model and tokenizer  
5  model = AutoModelForCausalLM.from_pretrained("gpt2")  
6  tokenizer = AutoTokenizer.from_pretrained("gpt2")  
7  
8  # Load and tokenize data  
9  dataset = load_dataset("wikitext", "wikitext-2-raw-v1")  
10 def tokenize(examples):  
11     return tokenizer(examples["text"], truncation=True,  
12                       max_length=512)  
12 tokenized = dataset.map(tokenize, batched=True)  
13
```

continued...

Training with HuggingFace

```
14 # Configure training
15 args = TrainingArguments(
16     output_dir="./results",
17     num_train_epochs=3,
18     per_device_train_batch_size=8,
19     learning_rate=5e-5,
20     warmup_steps=500,
21     weight_decay=0.01,
22     logging_steps=100,
23 )
24
25 # Train!
26 trainer = Trainer(model=model, args=args, train_dataset=tokenized["train"])
```

...continued...

Training with HuggingFace

27 | `trainer.train()`

...continued

Scaling laws

Kaplan et al. (2020): neural scaling laws

OpenAI discovered that language model performance follows **predictable power laws** as you increase:

1. **Model size** (N parameters): $\mathcal{L} \propto N^{-0.076}$
2. **Dataset size** (D tokens): $\mathcal{L} \propto D^{-0.095}$
3. **Compute budget** (C FLOPs): $\mathcal{L} \propto C^{-0.050}$

These relationships are remarkably smooth across **seven orders of magnitude**.

What this means

Performance improves as a straight line on a log-log plot. There are no sudden jumps or plateaus — just smooth, predictable improvement. This means we can *predict* how well a model will perform before training it.

Visualizing scaling laws

Log-log plots reveal power laws

When we plot loss vs. compute on log-log axes, the relationship is linear:

$$\log(\mathcal{L}) = -a \log(C) + \beta$$

This means doubling compute reduces loss by a *fixed percentage* — not a fixed amount.

Concrete numbers

1	Model	Parameters	Loss	Perplexity
2	GPT-2 small	117M	3.30	27.0
3	GPT-2 medium	345M	3.07	21.5
4	GPT-2 large	774M	2.93	18.8
5	GPT-2 XL	1.5B	2.85	17.4
6	GPT-3	175B	~2.4	~11.0

Each ~10x increase in parameters gives roughly the same *percentage* improvement in loss.

Chinchilla scaling

Hoffmann et al. (2022): compute-optimal training

Kaplan et al. suggested scaling model size faster than data. **Chinchilla** showed this was wrong:

For a fixed compute budget, the optimal allocation is approximately:

$$\text{tokens} \approx 20 \times \text{parameters}$$

A 70B parameter model should be trained on ~1.4 trillion tokens.

Many models were undertrained

Chinchilla (70B params, 1.4T tokens) matched GPT-3 (175B params, 300B tokens) despite being **2.5x smaller**. GPT-3 was undertrained relative to its size — it needed more data, not more parameters.

What scaling laws tell us

Practical implications

1. **Predictability:** We can estimate final performance from small-scale experiments — train small models first, then extrapolate
2. **Resource allocation:** Don't just make models bigger — balance parameters and data
3. **No free lunch:** Improving loss by 10% requires roughly 10x more compute
4. **Emergent abilities:** Some capabilities (like arithmetic, translation) appear suddenly at certain scales, even though the loss curve is smooth

The cost of scale

Model	Parameters	Training cost (est.)
BERT-base	110M	~\$5,000
GPT-2	1.5B	~\$50,000
GPT-3	175B	~\$4,600,000
GPT-4	~1.8T (rumored)	~\$100,000,000

Pre-training vs fine-tuning

Two phases of training

Pre-training: Train a large model on a massive, general-purpose corpus (e.g., internet text). The model learns language patterns, facts, and reasoning. This is expensive and done once.

Fine-tuning: Take the pre-trained model and train it further on a smaller, task-specific dataset. This adapts the general knowledge to a specific use case. This is cheap and done many times.

Analogy

Pre-training is like getting a liberal arts education — broad knowledge about many topics. Fine-tuning is like specializing in medical school — adapting that broad foundation to a specific domain.

Why does fine-tuning work?

Transfer learning

During pre-training, the model learns:

- Grammar and syntax
- World knowledge and facts
- Reasoning patterns
- Contextual understanding

These capabilities **transfer** to new tasks. Fine-tuning only needs to teach the model *how to apply* its existing knowledge to the new task format.

Key advantage

Fine-tuning a pre-trained model on 1,000 labeled examples typically outperforms training from scratch on 100,000 examples. The pre-trained representations give the model a massive head start.

Fine-tuning with HuggingFace

Fine-tuning BERT for sentiment classification

```
1 from transformers import AutoModelForSequenceClassification,  
  Trainer, TrainingArguments  
2 from datasets import load_dataset  
3  
4 # Load pre-trained BERT + add classification head  
5 model = AutoModelForSequenceClassification.from_pretrained(  
6     "bert-base-uncased", num_labels=2 # positive/negative  
7 )  
8  
9 # Load task-specific data  
10 dataset = load_dataset("imdb") # 50,000 movie reviews  
11
```

continued...

Fine-tuning with HuggingFace

```
12 # Fine-tuning hyperparameters (much smaller than pre-training!)
13 args = TrainingArguments(
14     output_dir="./sentiment-model",
15     num_train_epochs=3,          # just 3 epochs
16     per_device_train_batch_size=16,
17     learning_rate=2e-5,         # very small learning rate
18     warmup_ratio=0.1,
19     weight_decay=0.01,
20     evaluation_strategy="epoch",
21 )
22
```

...continued...

Fine-tuning with HuggingFace

```
23 | trainer = Trainer(model=model, args=args,  
24 |     train_dataset=dataset["train"], eval_dataset=dataset["test"])  
25 | trainer.train()  
26 | # Achieves ~93% accuracy in ~30 minutes on a single GPU!
```

...continued

Practical training advice

Rules of thumb

1. **Start small:** Test your pipeline with a small model before scaling up
2. **Learning rate:** Fine-tuning uses 10-100x smaller learning rates than pre-training (typically $1e-5$ to $5e-5$)
3. **Epochs:** Fine-tuning needs only 2-5 epochs (vs. 1 epoch for pre-training on large data)
4. **Batch size:** Larger batches give more stable gradients but require more memory. Use gradient accumulation if your GPU is too small
5. **Evaluation:** Always hold out a validation set and monitor for overfitting

Common training pitfalls

Things that go wrong

Overfitting: The model memorizes the training data instead of learning general patterns. Signs: training loss drops but validation loss increases. Fix: more data, dropout, early stopping.

Catastrophic forgetting: During fine-tuning, the model "forgets" its pre-trained knowledge. Fix: use small learning rates, freeze lower layers, or use techniques like LoRA.

Data quality: Garbage in, garbage out. Models trained on noisy, biased, or duplicated data learn those patterns. Fix: careful data curation and deduplication.

Training instability: Loss spikes or NaN values during training. Fix: gradient clipping, learning rate warmup, smaller batch sizes.

Discussion: scaling and efficiency

Questions to consider

1. If scaling laws predict smooth improvement, will we reach human-level performance by just scaling up? Why or why not?
2. Training GPT-4 reportedly cost ~\$100M. Is this sustainable? Who can afford to train these models?
3. Chinchilla showed we can match larger models with more data. What are the implications for open-source models?
4. How should we balance model performance against environmental cost (energy, carbon emissions)?

Discussion: training data and ethics

Questions to consider

1. Pre-training data comes from the internet — who gave permission? What about copyright?
2. Models learn biases present in training data (gender, race, culture). Whose responsibility is it to fix this?
3. Fine-tuning with "human feedback" (RLHF) introduces the biases of the annotators. How do we account for this?
4. Should training data be publicly documented? What are the tradeoffs of transparency?

References

Further reading

Kaplan et al. (2020, *arXiv*) "Scaling Laws for Neural Language Models" — Discovered power-law scaling relationships.

Hoffmann et al. (2022, *arXiv*) "Training Compute-Optimal Large Language Models" — The Chinchilla paper on optimal data-parameter balance.

Vaswani et al. (2017, *NeurIPS*) "Attention Is All You Need" — The original transformer paper.

HuggingFace NLP Course, Chapter 3 — Hands-on tutorial for fine-tuning pre-trained models.

Loshchilov & Hutter (2019, *ICLR*) "Decoupled Weight Decay Regularization" — The AdamW optimizer paper.

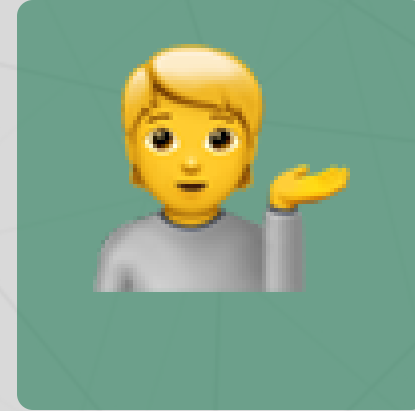
Questions?



Email



Discord



Office hours

Up next...

Retrieval Augmented Generation (RAG): giving language models access to external knowledge!